

Multiple Contacts

© James Wadsley, 2VG3

Reference Text:

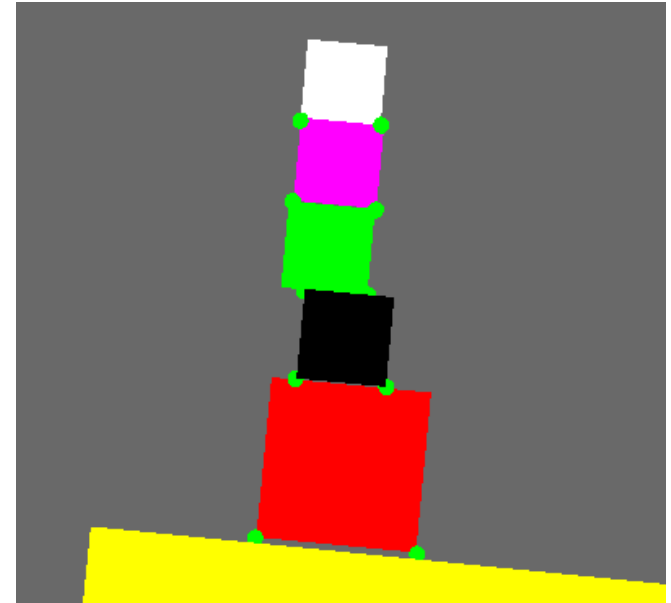
Ian Millington:

Game Physics Engine Development

Chapters 16

Multiple contacts

- We have tried to limit our selves to simple cases, e.g. point collisions
- In practice most engines need to deal with many contacts at once between many objects (e.g. piles and stacks)
- Even a face-face collision needs two contacts to be resolved properly



Collision Impulse with Rotation 2D

- General 2D collision must conserve linear momentum and energy including bulk kinetic and rotational energy
- Energy conserved according to value of e (coefficient of restitution)

$$\begin{aligned} m_1 \vec{v}_1 &= m_1 \vec{u}_1 + \Delta \vec{p} \\ m_2 \vec{v}_2 &= m_2 \vec{u}_2 - \Delta \vec{p} \\ I_1 \omega_1^{new} &= I_1 \omega_1 + \vec{r}_{P,1}' \times \Delta \vec{p} \\ I_2 \omega_2^{new} &= I_2 \omega_2 - \vec{r}_{P,2}' \times \Delta \vec{p} \end{aligned}$$

$$\begin{aligned} \Delta \vec{p} &= \frac{(1+e)((\vec{u}_{P,2} - \vec{u}_{P,1}) \cdot \hat{n}_{12})}{\left(\frac{1}{m_1} + \frac{1}{m_2}\right) + \frac{|\vec{r}_{P,1}' \times \hat{n}_{12}|^2}{I_1} + \frac{|\vec{r}_{P,2}' \times \hat{n}_{12}|^2}{I_2}} \hat{n}_{12} \\ \hat{n}_{12} &= \text{Collision Surface normal} \end{aligned}$$

Collision Impulse with Rotation 2D

- General 2D collision must conserve linear momentum and energy including bulk kinetic and rotational energy
- Energy conserved according to value of e (coefficient of restitution)

$$\begin{aligned} m_1 \vec{v}_1 &= m_1 \vec{u}_1 + \Delta \vec{p} \\ m_2 \vec{v}_2 &= m_2 \vec{u}_2 - \Delta \vec{p} \\ I_1 \omega_1^{new} &= I_1 \omega_1 + \vec{r}_{P,1}' \times \Delta \vec{p} \\ I_2 \omega_2^{new} &= I_2 \omega_2 - \vec{r}_{P,2}' \times \Delta \vec{p} \end{aligned}$$

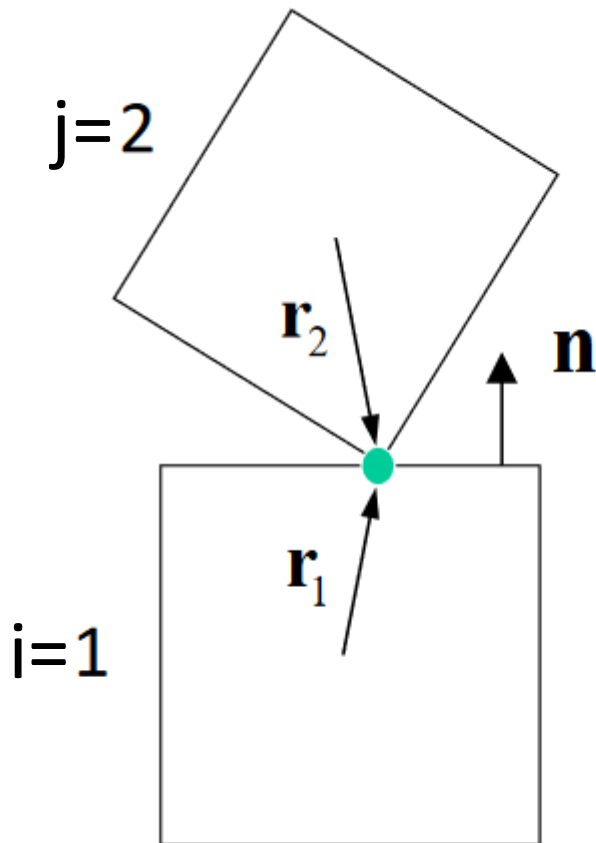
$$\begin{aligned} \Delta \vec{p} &= \frac{(1+e)((\vec{u}_{P,2} - \vec{u}_{P,1}) \cdot \hat{n}_{12})}{\left(\frac{1}{m_1} + \frac{1}{m_2}\right) + \frac{|\vec{r}_{P,1}' \times \hat{n}_{12}|^2}{I_1} + \frac{|\vec{r}_{P,2}' \times \hat{n}_{12}|^2}{I_2}} \hat{n}_{12} \\ \hat{n}_{12} &= \text{Collision Surface normal} \end{aligned}$$

Reminder: What is it actually doing?

Collisions

- For every contact, we have an initial relative normal velocity for the two points in contact that must be fixed if negative.

The new target normal velocity is $-e$ x the original

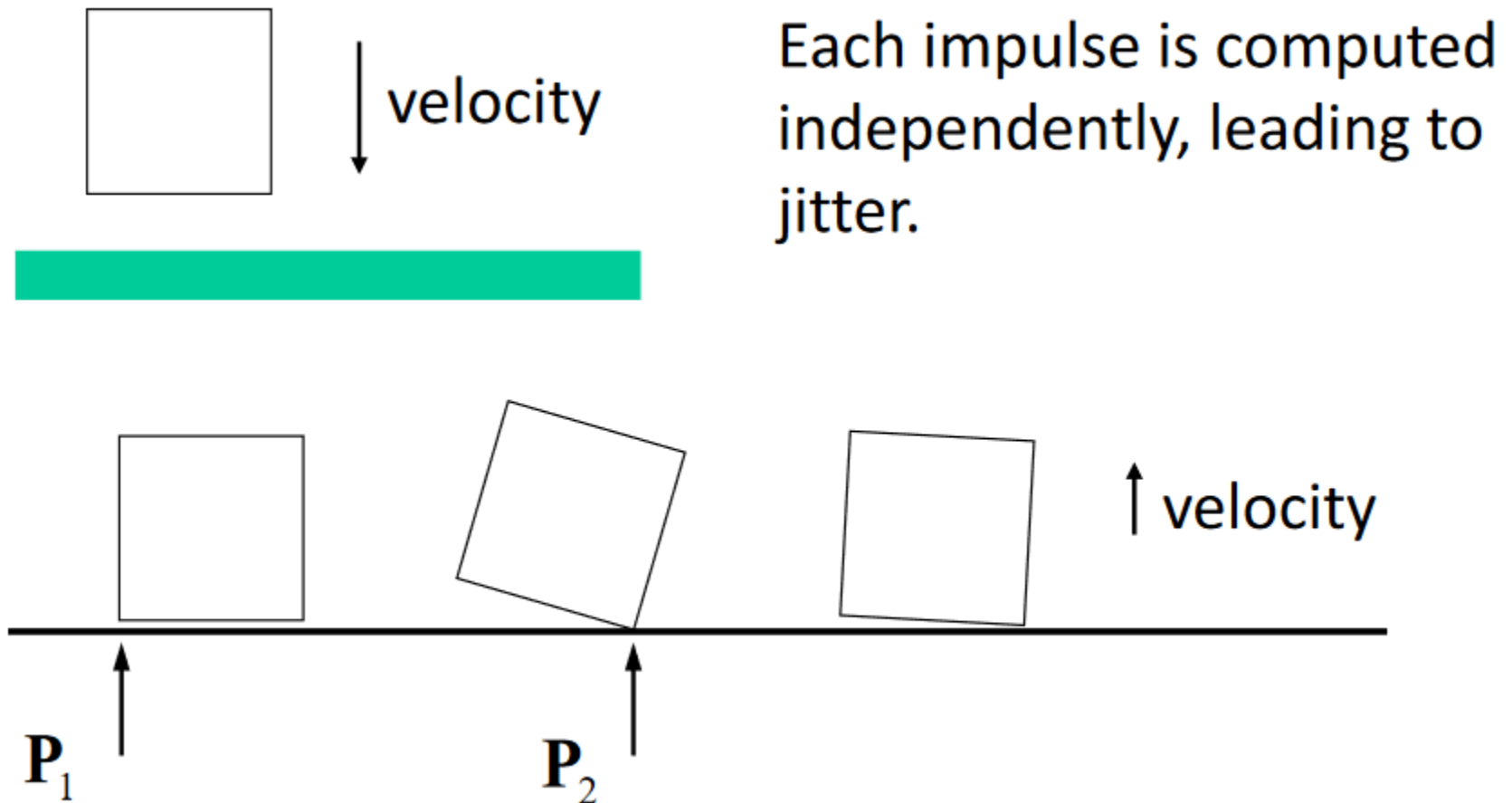


$$\Delta \mathbf{v} = \mathbf{v}_2 + \boldsymbol{\omega}_2 \times \mathbf{r}_2 - \mathbf{v}_1 - \boldsymbol{\omega}_1 \times \mathbf{r}_1$$

Before: $\Delta \vec{u}_n = (\vec{u}_{ij} \cdot \hat{n}_{ij}) \hat{n}_{ij}$

After: $\Delta \vec{v}_n = (\vec{v}_{ij} \cdot \hat{n}_{ij}) \hat{n}_{ij}$
 $= -e \Delta \vec{u}_n$

Multiple contacts



What's going wrong?

- The second and later contacts undo the fix for the first one: $\mathbf{v}_1$ $\mathbf{v}_2$ no longer satisfy the target!
- This is now a system of linear constraints
- It should be possible to solve all of them together with the right set of impulses

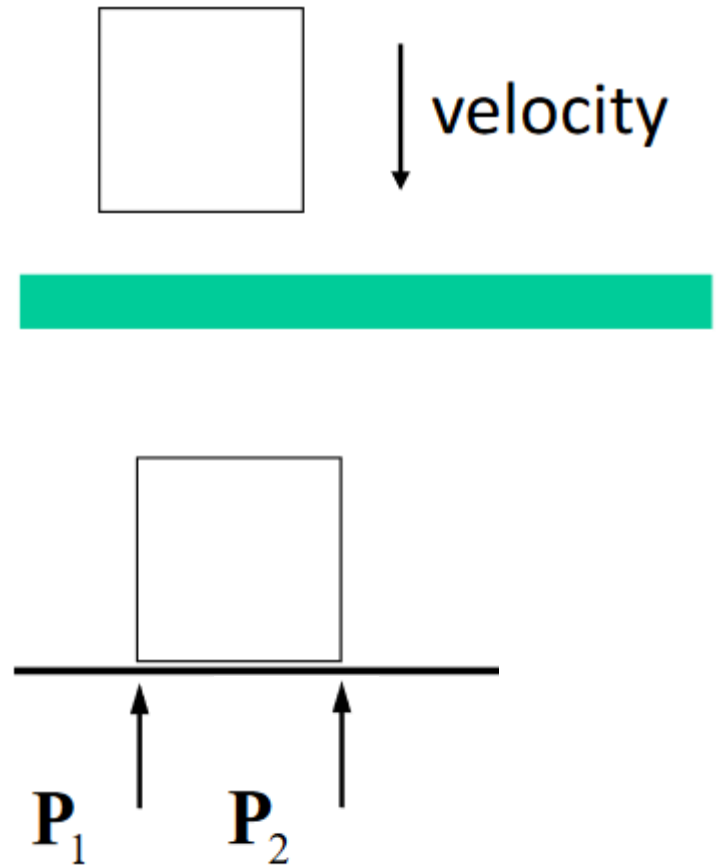
$$\Delta \mathbf{v} = \mathbf{v}_2 + \boldsymbol{\omega}_2 \times \mathbf{r}_2 - \mathbf{v}_1 - \boldsymbol{\omega}_1 \times \mathbf{r}_1$$

Before: $\Delta \vec{u}_n = (\vec{u}_{ij} \cdot \hat{n}_{ij}) \hat{n}_{ij}$

After: $\Delta \vec{v}_n = (\vec{v}_{ij} \cdot \hat{n}_{ij}) \hat{n}_{ij}$
 $= -e \Delta \vec{u}_n$

Our quick fix:

- For a simple collision we can average them together $P_{\text{eff}} = \frac{1}{2} (P_1 + P_2)$
- Does not work in general, e.g. for friction or multiple objects (Stacks)



Linear System version (illustration only – no matrix inversion required in this course)

$$\begin{aligned}\Delta \vec{v}_n &= (\vec{v}_{ij} \cdot \hat{n}_{ij}) \hat{n}_{ij} \\ &= \vec{v}_j + \vec{\omega}_j \times \vec{r}_{jc} - \vec{v}_i - \vec{\omega}_i \times \vec{r}_{ic} = -e \Delta \vec{u}_n\end{aligned}$$

- This a set of constants times the velocities and omegas ω . We can put it in matrix form. e.g. if $e=0$ (inelastic collision)

$\dot{C}: d = 0$
 $\Rightarrow \dot{C}: (\vec{v}_1 + w_1 \times \vec{r}_1 - \vec{v}_2 - w_2 \times \vec{r}_2) \cdot \vec{n} = 0$ This is the inelastic case ($e=0$): zero relative movement

$\Rightarrow \dot{C}: \begin{bmatrix} \vec{n}^T & \text{magnitude}(\vec{r}_1 \times \vec{n})^T & -\vec{n}^T & -\text{magnitude}(\vec{r}_2 \times \vec{n})^T \end{bmatrix} \begin{bmatrix} \vec{v}_1 \\ w_1 \\ \vec{v}_2 \\ w_2 \end{bmatrix} = 0$ Rewrite as a matrix equation

if $J = \begin{bmatrix} \vec{n}^T & \text{magnitude}(\vec{r}_1 \times \vec{n})^T & -\vec{n}^T & -\text{magnitude}(\vec{r}_2 \times \vec{n})^T \end{bmatrix}$, and $V = \begin{bmatrix} \vec{v}_1 \\ w_1 \\ \vec{v}_2 \\ w_2 \end{bmatrix}$ then,

$\dot{C}: JV = 0$,equation(1)

Solving for impulses $\lambda = \Delta p$

$$\text{if } V = \begin{bmatrix} \vec{v}_1 \\ w_1 \\ \vec{v}_2 \\ w_2 \end{bmatrix}, A = \begin{bmatrix} \vec{a}_1 \\ \alpha_1 \\ \vec{a}_2 \\ \alpha_2 \end{bmatrix}, M = \begin{bmatrix} m1 & 0 & 0 & 0 & 0 & 0 \\ 0 & m1 & 0 & 0 & 0 & 0 \\ 0 & 0 & I1 & 0 & 0 & 0 \\ 0 & 0 & 0 & m2 & 0 & 0 \\ 0 & 0 & 0 & 0 & m2 & 0 \\ 0 & 0 & 0 & 0 & 0 & I2 \end{bmatrix}, F = \begin{bmatrix} \vec{f}_1 \\ \tau_1 \\ \vec{f}_2 \\ \tau_2 \end{bmatrix}, \text{ then}$$

$$\Delta V = At$$

$$\Rightarrow \Delta V = \frac{F}{M}$$

$$\Rightarrow \Delta V = \frac{J^T \lambda}{M}$$

$$\Rightarrow \Delta V = M^{-1} J^T \lambda \dots \dots \text{equation(3)}$$

Next we substitute this to previous equation

$$J(V + \Delta V) = 0$$

$$\Rightarrow J(V + M^{-1} J^T \lambda) = 0$$

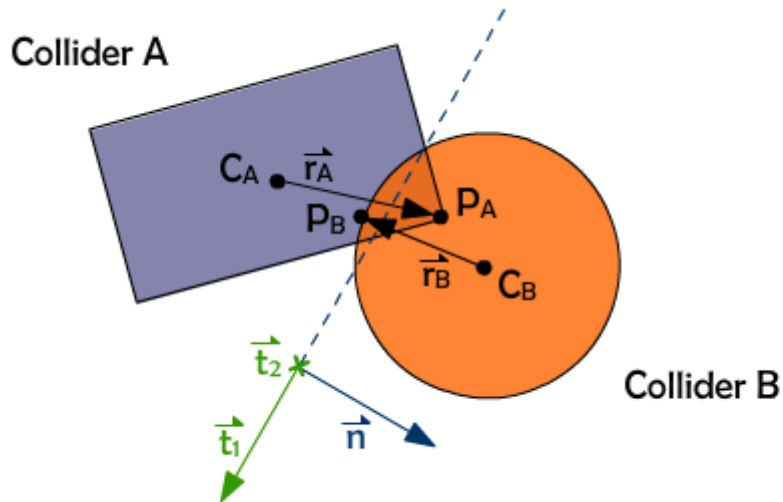
$$\Rightarrow \lambda = \frac{-(JV)}{JM^{-1}J^T} \dots \dots \dots \text{equation(4)}$$

For a given initial $v_1, \omega_1, v_2, \omega_2$
Solve for an impulse Δp that
Will solve the collision constraint
of zero relative velocity.

There is one unique solution that
is the same as our formula for a
single point collision.

What is new here is that we can
set up as many constraints at once
as we like – we just have to invert
the matrix.

Collisions as constraints



C Position Constraint

$$C : (P_B - P_A) \cdot \vec{n} \geq 0$$

$$C : (C_B + \vec{r}_B - C_A - \vec{r}_A) \cdot \vec{n} \geq 0$$

$\dot{C}$ Velocity Constraint

$$\dot{C} : (-\vec{V}_A - \vec{\omega}_A \times \vec{r}_A + \vec{V}_B + \vec{\omega}_B \times \vec{r}_B) \cdot \vec{n} \geq 0$$

If $C \geq 0$ initially, can ensure it always stays true over time using the derivative

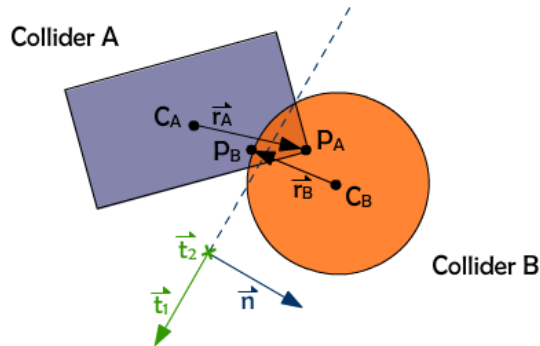
($\dot{C}$) version of the constraint:

Now $\dot{C} \geq 0$

Equivalent to requiring normal part of relative velocity is positive (moving apart)

Our impulse approach satisfies this constraint for one collision

Collisions as constraints



$$\dot{C} : (-\vec{V}_A - \vec{\omega}_A \times \vec{r}_A + \vec{V}_B + \vec{\omega}_B \times \vec{r}_B) \cdot \vec{n} \geq 0$$

$$J = [-\vec{n}^T \quad (-\vec{r}_A \times \vec{n})^T \quad \vec{n}^T \quad (\vec{r}_B \times \vec{n})^T],$$

$$V = \begin{bmatrix} \vec{V}_A \\ \vec{\omega}_A \\ \vec{V}_B \\ \vec{\omega}_B \end{bmatrix}$$

Restate this as a vector product

There is a range of possible velocities that satisfy the constraint.

For a specified coefficient of restitution $e=C_R$, there is one choice that dissipates the correct energy given by the expression on the right.

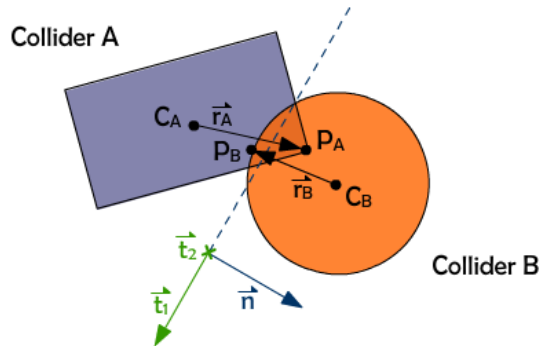
$$C_{\text{normal}} : J_n \cdot V \geq 0$$

$$C_{\text{normal}} : J_n \cdot V + b = 0$$

Note that $b < 0$ for approaching points A, B so $J \cdot V \geq 0$ as required

$$b = C_R(-\vec{V}_A - \vec{\omega}_A \times \vec{r}_A + \vec{V}_B + \vec{\omega}_B \times \vec{r}_B) \cdot \vec{n}$$

Collisions as constraints



$$\dot{C} : (-\vec{V}_A - \vec{\omega}_A \times \vec{r}_A + \vec{V}_B + \vec{\omega}_B \times \vec{r}_B) \cdot \vec{n} \geq 0$$

$$J = [-\vec{n}^T \quad (-\vec{r}_A \times \vec{n})^T \quad \vec{n}^T \quad (\vec{r}_B \times \vec{n})^T],$$

$$V = \begin{bmatrix} \vec{V}_A \\ \vec{\omega}_A \\ \vec{V}_B \\ \vec{\omega}_B \end{bmatrix}$$

What if already overlapping?

Moving to velocity space we have removed the penetration depth as a consideration. In general, for finite timesteps, we never catch collisions in time (so $d > 0$)

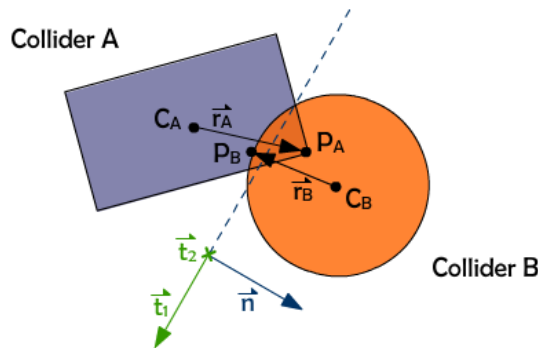
$$C_{\text{normal}} : J_n \cdot V + b = 0$$

$$d = (P_B - P_A) \cdot (-\vec{n})$$

We can adjust b to push the colliders apart using a bias term

$$b = -\frac{\beta}{\Delta t} \cdot d + C_R(-\vec{V}_A - \vec{\omega}_A \times \vec{r}_A + \vec{V}_B + \vec{\omega}_B \times \vec{r}_B) \cdot \vec{n}$$

Collisions as constraints: Bias vs. posFix



$$C_{\text{normal}} : J_n \cdot V + b = 0$$

$$d = (P_B - P_A) \cdot (-\vec{n})$$

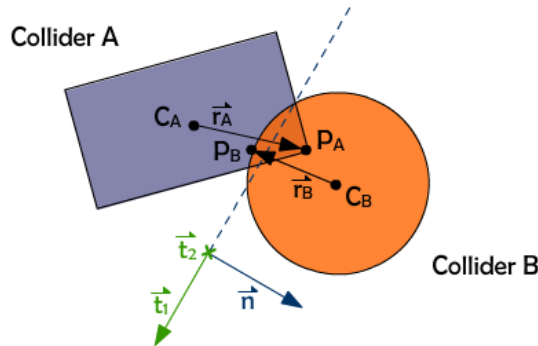
$$b = -\frac{\beta}{\Delta t} \cdot d + C_R(-\vec{V}_A - \vec{\omega}_A \times \vec{r}_A + \vec{V}_B + \vec{\omega}_B \times \vec{r}_B) \cdot \vec{n}$$

What if already overlapping?

Moving to velocity space we have removed the penetration depth as a consideration. In general, for finite timesteps, we never catch collisions in time (so $d > 0$)

1. We can adjust b to push the colliders apart using a bias term
This works well if we iterate carefully, requires trial and error $\beta \sim 0.1-0.3$
2. We can manually correct position (as done so far in rigidbody2d_fric.py etc...)

Collisions as constraints: Friction



$$\dot{C} : (-\vec{V}_A - \vec{\omega}_A \times \vec{r}_A + \vec{V}_B + \vec{\omega}_B \times \vec{r}_B) \cdot \vec{n} \geq 0$$

$$\dot{C} : JV + b \geq 0,$$

$$J = [-\vec{n}^T \quad (-\vec{r}_A \times \vec{n})^T \quad \vec{n}^T \quad (\vec{r}_B \times \vec{n})^T],$$

$$V = \begin{bmatrix} \vec{V}_A \\ \vec{\omega}_A \\ \vec{V}_B \\ \vec{\omega}_B \end{bmatrix}$$

Static friction works in the same framework. We need to constrain to have zero motion in the tangent direction t_1

We get a second constraint (in 2D) that this part of the velocity must be zero exactly

$$J_{t_1} = [-\vec{t}_1^T \quad (-\vec{r}_A \times \vec{t}_1)^T \quad \vec{t}_1^T \quad (\vec{r}_B \times \vec{t}_1)^T]$$

$$C_{\text{friction}} : J_{t_1} \cdot V = 0$$

Note: In 3D there is a third constraint, the point cannot move in the third dimension (t_2) either

Single Constraint

$$\lambda = \frac{-(JV + b)}{JM^{-1}J^T}$$

and M is the mass matrix:

$$M = \begin{bmatrix} M_A & 0 & 0 & 0 \\ 0 & I_A & 0 & 0 \\ 0 & 0 & M_B & 0 \\ 0 & 0 & 0 & I_B \end{bmatrix},$$

$$J = [-\vec{n}^T \quad (-\vec{r}_A \times \vec{n})^T \quad \vec{n}^T \quad (\vec{r}_B \times \vec{n})^T]$$

- the velocity correction is given by $\Delta V = M^{-1}J^T\lambda$
- Equivalent to our impulse solver:

$$\left. \begin{aligned} m_1 \vec{v}_1 &= m_1 \vec{u}_1 + \Delta \vec{p} \\ m_2 \vec{v}_2 &= m_2 \vec{u}_2 - \Delta \vec{p} \\ I_1 \omega_1^{new} &= I_1 \omega_1 + \vec{r}_{P,1}' \times \Delta \vec{p} \\ I_2 \omega_2^{new} &= I_2 \omega_2 - \vec{r}_{P,2}' \times \Delta \vec{p} \end{aligned} \right\} M \Delta V$$

$$V = \begin{bmatrix} \vec{V}_A \\ \vec{\omega}_A \\ \vec{V}_B \\ \vec{\omega}_B \end{bmatrix}$$

$$\Delta \vec{p} = \frac{(1+e)((\vec{u}_{P,2} - \vec{u}_{P,1}) \cdot \hat{n}_{12})}{\left(\frac{1}{m_1} + \frac{1}{m_2}\right) + \frac{|\vec{r}_{P,1}' \times \hat{n}_{12}|^2}{I_1} + \frac{|\vec{r}_{P,2}' \times \hat{n}_{12}|^2}{I_2}} \hat{n}_{12}$$

$\hat{n}_{12}$ = Collision Surface normal

Annotations in the diagram:

- A blue arrow points from the term $-(JV + b)$ to the numerator of the $\Delta \vec{p}$ equation.
- A blue arrow points from the term $JM^{-1}J^T$ to the denominator of the $\Delta \vec{p}$ equation.

Multiple Constraints

- Multiple constraints means multiple $\lambda_k = \Delta p_k$ to solve for
- This can be represented as a matrix with a new row for every contact

$$\begin{bmatrix} JM^{-1}J^T_1 \\ JM^{-1}J^T_2 \\ \dots \\ JM^{-1}J^T_n \end{bmatrix} \begin{bmatrix} \lambda_1 \\ \lambda_2 \\ \dots \\ \lambda_n \end{bmatrix} = \begin{bmatrix} -(JV + b)_1 \\ -(JV + b)_2 \\ \dots \\ -(JV + b)_n \end{bmatrix}$$

- Note: The M and V are the same if dealing with the same bodies but J will change depending on the contact point (normal and radius vectors different)
- For two bodies V is a 6 long vector in 2D (12 in 3D)
In general, however, V can also be lengthened to include as many bodies as needed
- You can also add in other kinds of constraints beyond collisions and expand the matrix as needed

Linear system solve

- The value of the matrix representation comes with multiple contacts: you can generate large matrices that include every contact (and potentially other kinds of constraints -- ball joints, hinges, sliders, etc...)
- Then you solve for all impulses at once dp_k for each contact k in some larger set $n > 1$
- There are general techniques to solve linear systems (invert matrices)

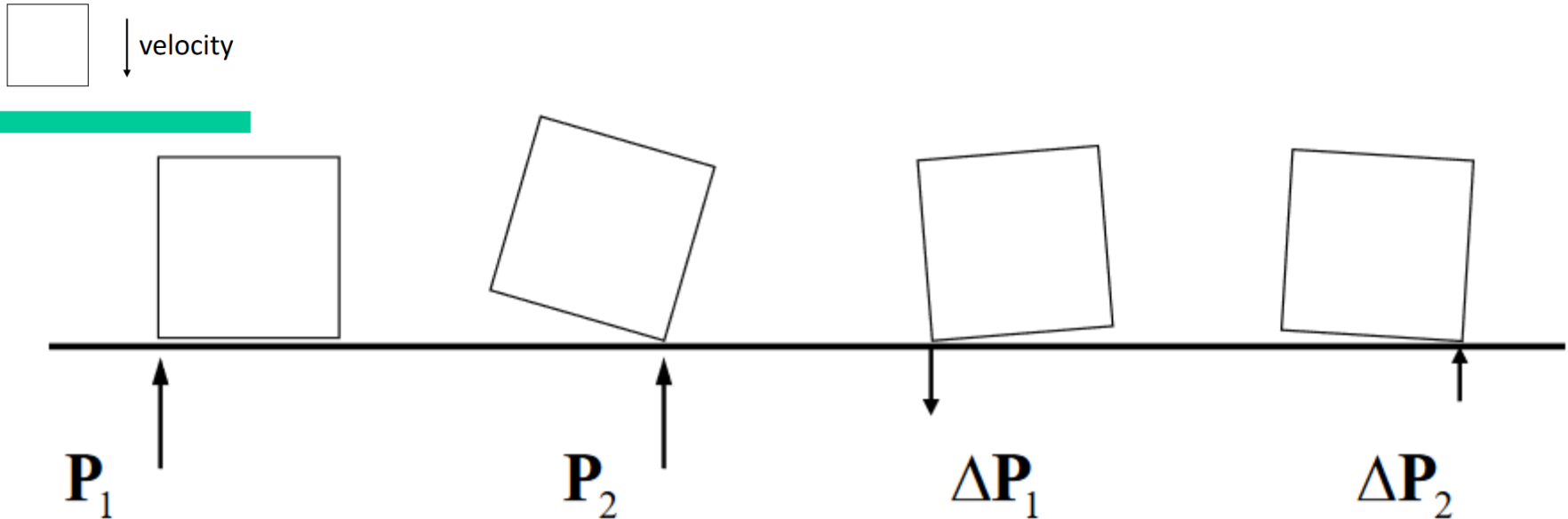
Linear System vs. Linear Complementarity Problem (LCP)

- We don't always have strict equality
- In particular, if two contacts are already moving apart OR another contact resolution makes them move apart, we don't need any added impulse there:
we are not allowed to apply net impulses that move the contacts towards each other!
- This makes it more complex than a normal linear solver (matrix problem)

$$\Delta p_k \leq 0$$

Sequential Impulses

- Inverting a matrix is expensive and tricky to also apply constraint to impulses at same time $\Delta p_k \leq 0$
- Erin Catto (Box2D, Angry Birds engine) suggested an iterative approach: sequentially apply corrections to the impulses to approximate the full solution (equivalent to solving the LCP)



Sequential Impulses: Algorithm

- Work out target normal velocity once

$$\begin{aligned}\Delta \vec{v}_n^{target} &= (\vec{v}_{ij} \cdot \hat{n}_{ij}) \hat{n}_{ij} \\ &= -e \Delta \vec{u}_n^{initial}\end{aligned}$$

- add small incremental impulses repeatedly to move overall impulse so that velocity goes to target

$$\begin{aligned}\Delta \vec{u}_n^{iter} &= (\vec{u}_{ij}^{iter} \cdot \hat{n}_{ij}) \hat{n}_{ij} \\ &= ((\vec{u}_j^{iter} + \vec{\omega}_j^{iter} \times \vec{r}_{jc} - \vec{u}_i^{iter} - \vec{\omega}_i^{iter} \times \vec{r}_{ic}) \cdot \hat{n}_{ij}) \hat{n}_{ij}\end{aligned}$$

$$\Delta \text{Impulse } \Delta \vec{p}^{iter} = \frac{\Delta \vec{u}_n^{iter} - \Delta \vec{v}_n^{target}}{\left(\frac{1}{m_1} + \frac{1}{m_2}\right) + \frac{|\vec{r}_{P,1}' \times \hat{n}_{12}|^2}{I_1} + \frac{|\vec{r}_{P,2}' \times \hat{n}_{12}|^2}{I_2}} \hat{n}_{12}$$

$$\hat{n}_{12} = \text{Collision Surface normal}$$

$$m_1 \vec{u}_1^{iter+1} = m_1 \vec{u}_1^{iter} + \Delta \vec{p}^{iter}$$

$$m_2 \vec{u}_2^{iter+1} = m_2 \vec{u}_2^{iter} - \Delta \vec{p}^{iter}$$

$$I_1 \omega_1^{iter+1} = I_1 \omega_1^{iter} + \vec{r}_{P,1}' \times \Delta \vec{p}^{iter}$$

$$I_2 \omega_2^{iter+1} = I_2 \omega_2^{iter} - \vec{r}_{P,2}' \times \Delta \vec{p}^{iter}$$

Sequential Impulses: Algorithm

- First iteration equivalent to old single impulse

$$\begin{aligned}\Delta \vec{v}_n^{target} &= (\vec{v}_{ij} \cdot \hat{n}_{ij}) \hat{n}_{ij} \\ &= -e \Delta \vec{u}_n^{initial}\end{aligned}$$

$$\begin{aligned}\Delta \vec{u}_n^{iter} &= (\vec{u}_{ij}^{iter} \cdot \hat{n}_{ij}) \hat{n}_{ij} \\ &= ((\vec{u}_j^{iter} + \vec{\omega}_j^{iter} \times \vec{r}_{jc} - \vec{u}_i^{iter} - \vec{\omega}_i^{iter} \times \vec{r}_{ic}) \cdot \hat{n}_{ij}) \hat{n}_{ij}\end{aligned}$$

$$\Delta \vec{p}^{iter} = \frac{\Delta \vec{u}_n^{iter} - \Delta \vec{v}_n^{target}}{\left(\frac{1}{m_1} + \frac{1}{m_2}\right) + \frac{|\vec{r}_{P,1}' \times \hat{n}_{12}|^2}{I_1} + \frac{|\vec{r}_{P,2}' \times \hat{n}_{12}|^2}{I_2}} \hat{n}_{12}$$

$$\Delta \vec{p} = \frac{(1+e)((\vec{u}_{P,2} - \vec{u}_{P,1}) \cdot \hat{n}_{12})}{\left(\frac{1}{m_1} + \frac{1}{m_2}\right) + \frac{|\vec{r}_{P,1}' \times \hat{n}_{12}|^2}{I_1} + \frac{|\vec{r}_{P,2}' \times \hat{n}_{12}|^2}{I_2}} \hat{n}_{12}$$

$$\hat{n}_{12} = \text{Collision Surface normal}$$

$$m_1 \vec{u}_1^{iter+1} = m_1 \vec{u}_1^{iter} + \Delta \vec{p}^{iter}$$

$$m_2 \vec{u}_2^{iter+1} = m_2 \vec{u}_2^{iter} - \Delta \vec{p}^{iter}$$

$$I_1 \omega_1^{iter+1} = I_1 \omega_1^{iter} + \vec{r}_{P,1}' \times \Delta \vec{p}^{iter}$$

$$I_2 \omega_2^{iter+1} = I_2 \omega_2^{iter} - \vec{r}_{P,2}' \times \Delta \vec{p}^{iter}$$

$$m_1 \vec{v}_1 = m_1 \vec{u}_1 + \Delta \vec{p}$$

$$m_2 \vec{v}_2 = m_2 \vec{u}_2 - \Delta \vec{p}$$

$$I_1 \omega_1^{new} = I_1 \omega_1 + \vec{r}_{P,1}' \times \Delta \vec{p}$$

$$I_2 \omega_2^{new} = I_2 \omega_2 - \vec{r}_{P,2}' \times \Delta \vec{p}$$

Sequential Impulses: Algorithm

- What's changes in the outcome with iterations?

$$\begin{aligned}\Delta \vec{v}_n^{target} &= (\vec{v}_{ij} \cdot \hat{n}_{ij}) \hat{n}_{ij} \\ &= -e \Delta \vec{u}_n^{initial}\end{aligned}$$

$$\begin{aligned}\Delta \vec{u}_n^{iter} &= (\vec{u}_{ij}^{iter} \cdot \hat{n}_{ij}) \hat{n}_{ij} \\ &= ((\vec{u}_j^{iter} + \vec{\omega}_j^{iter} \times \vec{r}_{jc} - \vec{u}_i^{iter} - \vec{\omega}_i^{iter} \times \vec{r}_{ic}) \cdot \hat{n}_{ij}) \hat{n}_{ij}\end{aligned}$$

$$\Delta \vec{p}^{iter} = \frac{\Delta \vec{u}_n^{iter} - \Delta \vec{v}_n^{target}}{\left(\frac{1}{m_1} + \frac{1}{m_2}\right) + \frac{|\vec{r}_{P,1}' \times \hat{n}_{12}|^2}{I_1} + \frac{|\vec{r}_{P,2}' \times \hat{n}_{12}|^2}{I_2}} \hat{n}_{12}$$

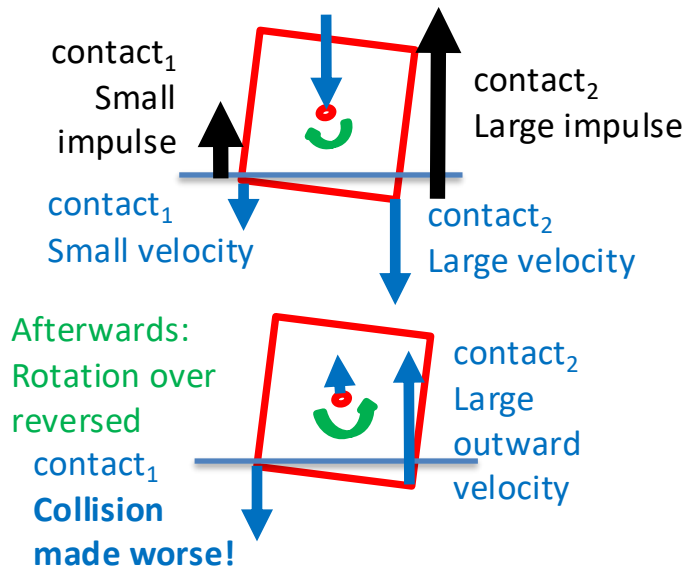
$$\begin{aligned}\Delta \vec{p} &= \frac{(1+e)((\vec{u}_{P,2} - \vec{u}_{P,1}) \cdot \hat{n}_{12})}{\left(\frac{1}{m_1} + \frac{1}{m_2}\right) + \frac{|\vec{r}_{P,1}' \times \hat{n}_{12}|^2}{I_1} + \frac{|\vec{r}_{P,2}' \times \hat{n}_{12}|^2}{I_2}} \hat{n}_{12} \\ \hat{n}_{12} &= \text{Collision Surface normal}\end{aligned}$$

$$\begin{aligned}m_1 \vec{u}_1^{iter+1} &= m_1 \vec{u}_1^{iter} + \Delta \vec{p}^{iter} \\ m_2 \vec{u}_2^{iter+1} &= m_2 \vec{u}_2^{iter} - \Delta \vec{p}^{iter} \\ I_1 \omega_1^{iter+1} &= I_1 \omega_1^{iter} + \vec{r}_{P,1}' \times \Delta \vec{p}^{iter} \\ I_2 \omega_2^{iter+1} &= I_2 \omega_2^{iter} - \vec{r}_{P,2}' \times \Delta \vec{p}^{iter}\end{aligned}$$

$$\begin{aligned}m_1 \vec{v}_1 &= m_1 \vec{u}_1 + \Delta \vec{p} \\ m_2 \vec{v}_2 &= m_2 \vec{u}_2 - \Delta \vec{p} \\ I_1 \omega_1^{new} &= I_1 \omega_1 + \vec{r}_{P,1}' \times \Delta \vec{p} \\ I_2 \omega_2^{new} &= I_2 \omega_2 - \vec{r}_{P,2}' \times \Delta \vec{p}\end{aligned}$$

Sequential Impulses: How it works

Two single impulses



Old approach:
Ad hoc combination of
single impulses

Outcome bad: collision not resolved!

Goal: velocities should be exactly flipped

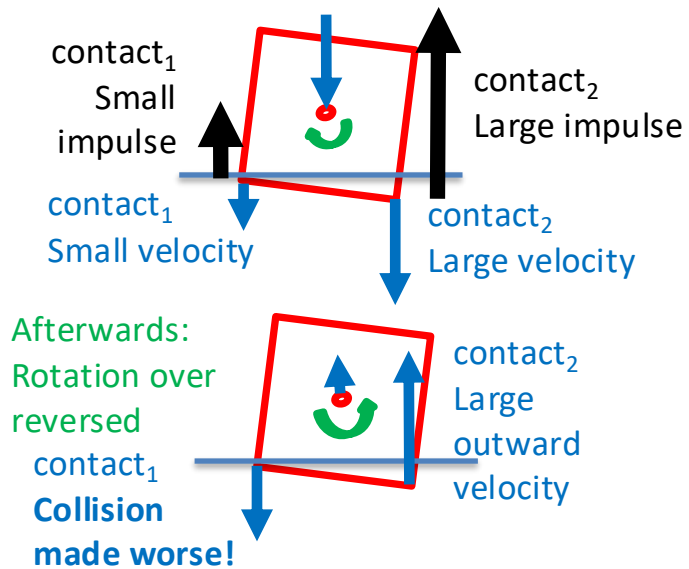
For $e=1$ (elastic collisions)

Rotation from contact₁ impulse pushed
contact₂ in faster!

Energy not conserved because rotational
energy of impulses partly cancel

Sequential Impulses: How it works

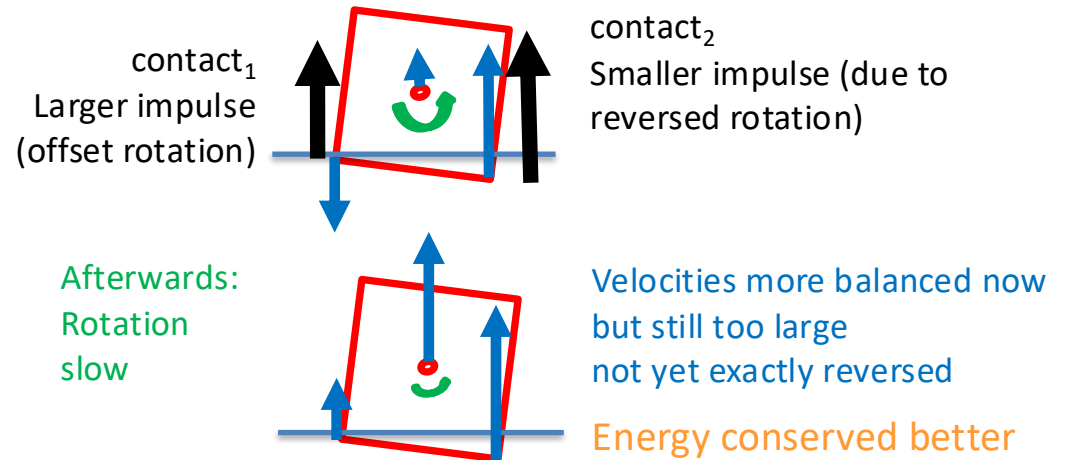
Two single impulses → 1st sequential impulse



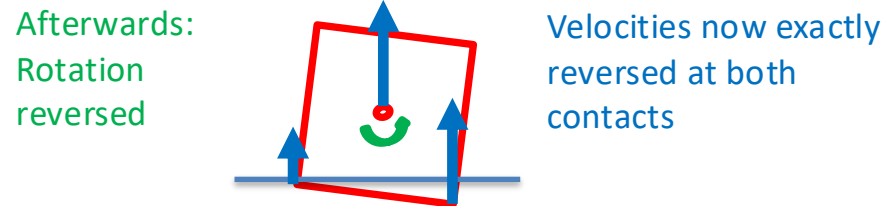
Energy lost

Now, keep going ...

Correct impulses 2nd sequential impulse iteration



After 100 iterations ...



Approaches perfect outcome
Energy conserved

Sequential Impulses: Algorithm

$$\begin{aligned}\Delta \vec{v}_n^{target} &= (\vec{v}_{ij} \cdot \hat{n}_{ij}) \hat{n}_{ij} \\ &= -e \Delta \vec{u}_n^{initial}\end{aligned}$$

$$\begin{aligned}\Delta \vec{u}_n^{iter} &= (\vec{u}_{ij}^{iter} \cdot \hat{n}_{ij}) \hat{n}_{ij} \\ &= ((\vec{u}_j^{iter} + \vec{\omega}_j^{iter} \times \vec{r}_{jc} - \vec{u}_i^{iter} - \vec{\omega}_i^{iter} \times \vec{r}_{ic}) \cdot \hat{n}_{ij}) \hat{n}_{ij}\end{aligned}$$

$$\Delta \text{ Impulse } \Delta \vec{p}^{iter} = \frac{\Delta \vec{u}_n^{iter} - \Delta \vec{v}_n^{target}}{\left(\frac{1}{m_1} + \frac{1}{m_2}\right) + \frac{|\vec{r}_{P,1}' \times \hat{n}_{12}|^2}{I_1} + \frac{|\vec{r}_{P,2}' \times \hat{n}_{12}|^2}{I_2}} \hat{n}_{12}$$

$$\hat{n}_{12} = \text{Collision Surface normal}$$

$$\text{Total } \Delta \vec{P}^{iter} = \sum_{iter} \Delta \vec{p}^{iter}$$

If $\text{Total } \Delta \vec{P}^{iter} > 0$ then

set $\text{Total } \Delta \vec{P}^{iter} = 0$ and

reset $\Delta \vec{p}^{iter} = \text{Total } \Delta \vec{P}^{iter} - \text{Total } \Delta \vec{P}^{iter-1}$

$$m_1 \vec{u}_1^{iter+1} = m_1 \vec{u}_1^{iter} + \Delta \vec{p}^{iter}$$

$$m_2 \vec{u}_2^{iter+1} = m_2 \vec{u}_2^{iter} - \Delta \vec{p}^{iter}$$

$$I_1 \omega_1^{iter+1} = I_1 \omega_1^{iter} + \vec{r}_{P,1}' \times \Delta \vec{p}^{iter}$$

$$I_2 \omega_2^{iter+1} = I_2 \omega_2^{iter} - \vec{r}_{P,2}' \times \Delta \vec{p}^{iter}$$

Previously we only treated contacts based on initial relative velocity < 0

Now they all stay in the mix and we enforce total impulse < 0 instead

(Only pushing apart allowed)

Note: These are equivalent for a single impulse

Sequential Impulses: Friction

- Previously Friction was quite approximate
- Now we can enforce a hard constraint that contacting points do not move.
- If we have two or more contacts, this means the body *cannot move at all*
(Note: assuming we iterate enough to get an accurate set of impulses)

Big benefit of iterative solver (e.g. accurate impulses) is can prevent jitter, keep things static

Sequential Impulses: Friction

$$\Delta \vec{v}_{//}^{target} = 0$$

$$\Delta \vec{u}_{//}^{iter} = \vec{u}_{ij}^{iter} - (\vec{u}_{ij}^{iter} \cdot \hat{n}_{ij}) \hat{n}_{ij}$$

$$\Delta Impulse \Delta \vec{p}_{Fric}^{iter} = \frac{\Delta \vec{u}_{//}^{iter}}{\left(\frac{1}{m_1} + \frac{1}{m_2} \right) + \frac{|\vec{r}_{P,1}' \times \hat{n}|^2}{I_1} + \frac{|\vec{r}_{P,2}' \times \hat{n}|^2}{I_2}}$$

$$\hat{n} = \text{Normal in direction of } \Delta \vec{u}_{//}$$

$$m_1 \vec{u}_1^{iter+1} = m_1 \vec{u}_1^{iter} + \Delta \vec{p}_{Fric}^{iter}$$

$$m_2 \vec{u}_2^{iter+1} = m_2 \vec{u}_2^{iter} - \Delta \vec{p}_{Fric}^{iter}$$

$$I_1 \omega_1^{iter+1} = I_1 \omega_1^{iter} + \vec{r}_{P,1}' \times \Delta \vec{p}_{Fric}^{iter}$$

$$I_2 \omega_2^{iter+1} = I_2 \omega_2^{iter} - \vec{r}_{P,2}' \times \Delta \vec{p}_{Fric}^{iter}$$

$$Total \Delta \vec{P}_{Fric}^{iter} = \sum_{iter} \Delta \vec{p}_{Fric}^{iter}$$

If $|Total \Delta \vec{P}_{Fric}^{iter}| > \mu_{static} |Total \Delta \vec{P}_{normal}|$ then

set $Total \Delta \vec{P}_{Fric}^{iter} = \mu_{kinetic} |Total \Delta \vec{P}_{normal}| \hat{n}$ and

set $\Delta \vec{p}_{Fric}^{iter} = Total \Delta \vec{P}_{Fric}^{iter} - Total \Delta \vec{P}_{Fric}^{iter-1}$

Implementing Sequential Impulses

Base Code

- Start with any recent collision with friction that is working:
e.g. `rigidbody2d_rolling.py`
Copy to:
`rigidbody2d_seq.py`
- You can start from:
`rigidbody2d_seq0.py`
from avenue. I have added comments to this file to show where changes are needed

Implementation key steps

- Store old position and orientation
Get new position, angle estimate for contact purposes
- Stored Variables needed (per contact):
 - Δp_{normal} Total Normal impulse (scalar)
 - Δp_{Fric} Total Frictional Impulse (vector)
 - Target Normal velocity (scalar)
- Need to iterate and update impulses (for loop)
 - Need to consider all contacts each iteration
- No longer need to test $u_{ij} \cdot n_{ij} < 0$ to define contact
- Calculate per iteration
 - change in Δp_{normal} (then correct so Total $p_{\text{normal}} < 0$)
 - change in Δp_{Fric} (then correct if Total $p_{\text{Fric}} > \mu_s$ normal impulse)
- No longer apply ad hoc position corrections
 - Throw away first position/angle update
 - Apply corrected velocities and rotation to original position and orientation
- Add v_{bias} to help positions

Store old velocity carefully!

- Python does not copy objects properly, it often just uses pointers

```
for p in self.RigidBody2Ds:

    a = Vec3(0.,0.,0.) # simple a here

    if (p.doGravity): a = Vec3(0.,0.,-self.g)

    p.vold = copy.deepcopy(p.vel) #NEW need a deepcopy or p.vold is
just a pointer to p.vel!
```

Use low e (Coefficients of restitution)

- If you have cylinders, go back to cubes for this testing

```
self.eCoeffRestitution = 0.05 #NEW
```

```
self.muFrictionStatic = 0.2
```

```
self.muFrictionKinetic = 0.2
```

Restore objects to squares/cubes

- If you have cylinders, go back to cubes for testing

```
self.RigidBody2Ds[0].shape = "cylinder" #REMOVE: square is default
```

```
self.tube.instanceTo(self.RigidBody2Ds[0]) #REMOVE: back to the cube
```

```
#Note: if your collideShapes doesn't work, you could use collideSquareSquare
```

```
contacts = collideSquareSquare( self, xi, xj, pi.radius, pj.radius, pi.theta, pj.theta )
```

Don't need to check current $u.n < 0$

- While we are iterating, the contact velocities may still be moving together or apart, we still include that contact

```
uij = uj-ui  
nij = normal #NEW  
unormal = uij.dot(nij) #NEW  
#check 3: must be approaching  
#if (uij.dot(normal) < 0.): #GONE  
if (True): #NEW
```

Set up on first iteration: total impulse (normal. friction) & vnormal

- We now apply impulses as small delta's every single iteration

However, we need to store (and update)

- Target velocity (how much it should bounce)
- Total impulse: normal (Δp_{Normal}) and friction (Δp_{Fric})
These are needed to ensure the forces are pushing apart only (Normal) and the Friction does not exceed μ_{static}
 $|\Delta p_{\text{Normal}}|$

```
#NEW iterate many times
niter = 100
dpNormal = [] #store total normal impulse
dpFric = []   #store total frictional impulse (parallel)
vnormal = []  #save original normal velocity at contact

for iter in range(niter): #NEW
    nContact = 0 # NEW
```

Set up on first iteration: total impulse (normal. friction) & vnormal

- We now apply impulses as small delta's every single iteration

However, we need to store (and update)

- Target velocity (how much it should bounce)
- Total impulse: normal (Δp_{Normal}) and friction (Δp_{Fric})
These are needed to ensure the forces are pushing apart only (Normal) and the Friction does not exceed μ_{static}
 $|\Delta p_{\text{Normal}}|$

```
if (iter==0):  
    dpNormal.append( 0. )  
    dpFric.append( Vec3(0,0,0) )  
  
    #BIAS -- avoid sinking into surface  
    vBias = dclose*0.3/dt  
  
    vNormalTarget.append(-self.eCoeffRestitution*uNormal+vBias)
```

Normal Impulse

- Modified calculation: get a delta impulse and correct if total impulse attractive (must be pushing apart)

```
#resolve collision
nij = normal
dxi = (xclose-xi)
dxj = (xclose-xj)
rni = dxi.cross(nij)
rnj = dxj.cross(nij)

iMFac = 1/(pi.inverseMass+pj.inverseMass + \
rni.dot(rni)*pi.inverseMomentOfInertia + \
rnj.dot(rnj)*pj.inverseMomentOfInertia)
#OLD: dp = nij*(1+self.eCoeffRestitution)*uij.dot(nij)*iMFac
# NEW code for dpNormal
dpNormalDelta = (unormal-vnormal[nContact])*iMFac
dpNormalNew = dpNormal[nContact] + dpNormalDelta
if (dpNormalNew > 0.): dpNormalNew = 0.
dpNormalDelta = dpNormalNew - dpNormal[nContact]
dpNormal[nContact] = dpNormalNew
```

Friction Impulse

- Modified calculation: get a delta impulse for Friction and correct if not static later now

```
# Code to calculate impulse parallel to surface

FricVecij = uij-nij*uij.dot(nij)
FricDir = FricVecij.normalized() #NEW
#New code for dpFric
up
rnfi = dxi.cross(FricDir) #equivalent to dxi.cross(dp)/|dp| friction will mess that
rnfj = dxj.cross(FricDir)
iMFacFric = 1/(pi.inverseMass+pj.inverseMass + \
              rnfi.dot(rnfi)*pi.inverseMomentOfInertia + \
              rnfj.dot(rnfj)*pj.inverseMomentOfInertia)
dpFricDelta = FricVecij*iMFacFric #effectively as above but target delta v // = 0
dpFricNew = dpFric[nContact] + dpFricDelta
FricRatio = dpFricNew.length()/(abs(dpNormalNew)+1e-20)
if (FricRatio > self.muFrictionStatic):
    dpFricNew = dpFricNew*self.muFrictionKinetic/FricRatio
    FricRatio = self.muFrictionKinetic
    dpFricDelta = dpFricNew - dpFric[nContact]

dpFric[nContact] = dpFricNew
```

Applying Impulse

- Apply delta impulse for every contact, for every iteration. Remove old updating code

```
#NEW update velocity and rotation each iteration

pi.vel += (nij*dpNormalDelta+dpFricDelta)*pi.inverseMass
pj.vel -= (nij*dpNormalDelta+dpFricDelta)*pj.inverseMass

pi.omega +=
(dxj.cross(nij*dpNormalDelta+dpFricDelta)).y*pi.inverseMomentOfInertia
pj.omega -=
(dxj.cross(nij*dpNormalDelta+dpFricDelta)).y*pj.inverseMomentOfInertia

#NEW advance to next contact after all changes done
nContact += 1
```

Final position update

- We changed positions (updated velocity with gravity and moved positions and angles)
- We now consider this provisional -- the only purpose is to find contacts
- We need to fix the velocity to resolve collisions (and any other constraints coming up ...)
- We literally throw away the provisional position and angle update and apply final iterated version averaged with vold

```
# Final position update

if (nContact > 0):
    if (self.fixPos):
        for p in self.RigidBody2Ds:
            p.setPos(p.rold + (p.vel+p.vold)*0.5*dt)
            p.theta = p.thetaold + p.omega*dt
```

Achtung!

Some possible bugs in your
sequential impulses

Set up on first iteration: total impulse (normal. friction) & vnormal

- Make sure not to make any change on the first iteration through -- just continue
- Otherwise the second contact will not record the correct vNormalTarget!

```
if (iter==0):  
    dpNormal.append( 0. )  
    dpFric.append( Vec3(0,0,0) )  
  
    #BIAS -- avoid sinking into surface  
    vBias = dclose*0.3/dt  
  
    vNormalTarget.append(-self.eCoeffRestitution*uNormal+vBias)  
    continue
```

Update nContact

- Must update nContact to work through them all and ensure $nContact > 0$ at the end

```
#Apply delta impulse

pi.vel += (nij*dpNormalDelta+dpFricDelta)*pi.inverseMass
pj.vel -= (nij*dpNormalDelta+dpFricDelta)*pj.inverseMass

pi.omega +=
(dxj.cross(nij*dpNormalDelta+dpFricDelta)).y*pi.inverseMomentOfInertia
pj.omega -=
(dxj.cross(nij*dpNormalDelta+dpFricDelta)).y*pj.inverseMomentOfInertia

nContact += 1    #replaces ncontacts
```

Alignment of indentation!

- Make sure the velocity update is only done right at the end:
After all iterations are over!

```
? def updateRigidBody2Ds(self, task):
    self.cleanMarkers()

    #use precise timesteps
    dt = self.dt
    self.t += self.dt
    self.iFrame += 1

    for p in self.RigidBody2Ds:
        a = Vec3(0.,0.,0.) # simple a here
        if (p.doGravity): a = Vec3(0.,0.,-self.g)
        p.vold = p.vel
        p.vel = p.vel + a*dt
        p.roid = p.getPos()
        r = p.roid + p.vel * dt
        p.setPos(r)
        #FIX: save old angle
        p.thetaold = p.theta
        p.theta += p.omega * dt
        p.setHpr( 90., p.theta*180/3.141592653589793238462, 0. )

    #FIX: add variables for dpNormal, dpFric, vNormalTarget
    dpNormal = []
    dpFriction = []
    vNormalTarget = []
    #FIX: Add for loop to iterate
    niter = 100
    for iter in range(niter):
        ncontact = 0
        #check other RigidBody2Ds in pairs (exclude self)
        . . .

    #FIX reapply new, corrected velocities to old position, old angle
    #Throw away initial update to position, angle
    if(ncontact > 0): JW moved left one tab
        for p in self.RigidBody2Ds:
            p.setPos(p.roid + p.vel*dt)
            p.theta = p.thetaold + p.omega*dt

    return task.cont

scene=SimpleScene()
scene.run()
```

Velocity Bias

- Box2D's Erin Catto recommends a bias parameter of 0.1-0.3.
- You may get better results outside this range (I do better with much smaller values or even 0)
- They also recommend turning it off for small penetration depths (see below)
- There may be other ideas that work. The best ideas would not need to be tweaked for different test cases.

```
vBias = dclose*0.3/dt
```

OR

```
vBias = 0 if (dclose<0.005) else dclose*0.3/dt
```

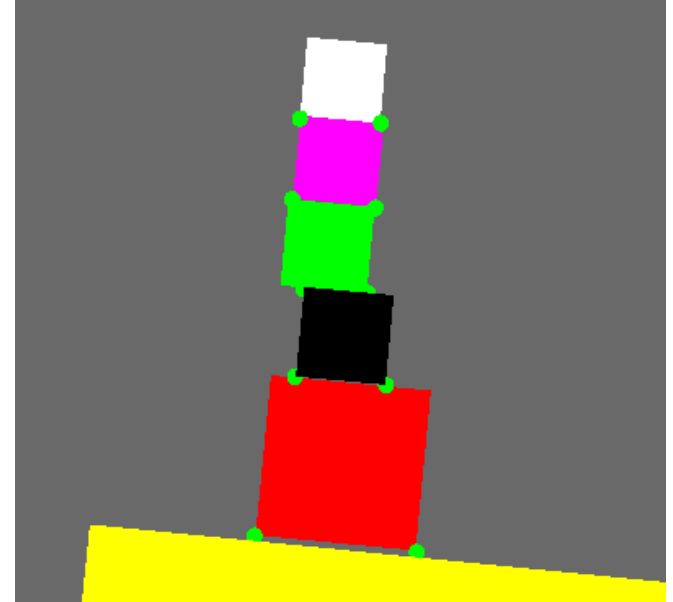
OR

?

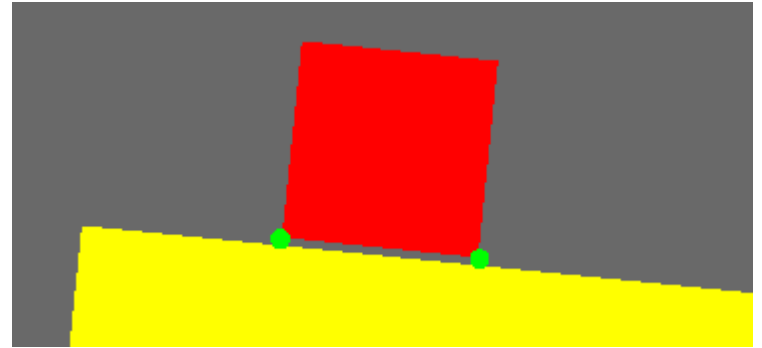
Optimization & Generalization: Frameworks for Contacts

Contact Manager:

- Find contacts once
- Keep constraints across many steps (if still applicable), save work on geometry
- Save work on iterations with warm start:
Start with prior impulse instead of basic guess:
fewer iterations to get new answer



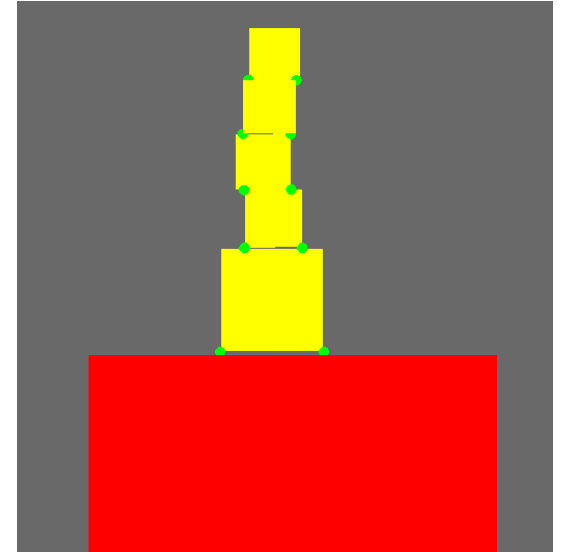
Testing: Inclined Plane

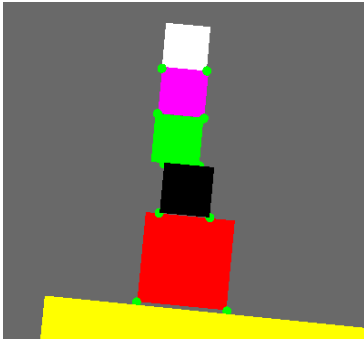


- Basic test case:
block on inclined plane with high static friction
- Compare old:
`rigidbody2d_friction.py`
to new:
`rigidbody2d_seq.py`
- Now should no longer slide with friction on, should always bounce cleanly for multiple contact collisions
May be some small visual jitter
(sensitive to choices: bias parameter, coefficient of restitution, number of iterations)

Testing: Stacking

- Stacking fails when the impulses conflict with each other
- If you iterate enough times (close enough to the exact answer), sequential impulses can handle stacking (e.g. Box2D). This can be expensive
- Try different set ups: flat and inclined, consider tweaking parameters like coefficient of restitution (low ~ 0.05), iterations and bias ($\beta \sim 0.1$ or less)





Testing: Stacking

```
#Initialize RigidBody2Ds
self.RigidBody2Ds = []
for i in range(6):
    p = RigidBody2D("p")
    p.doGravity = False
    p.inverseMass = 1.0
    self.cube.instanceTo(p)

    p.setScale(p.radius)
    p.setColor(Vec3(1-(i&2)//2,i&1,i&4))
    p.reparentTo(render)
    self.RigidBody2Ds.append(p)
```

Works best with lower
coefficient of friction, e.g.
e=0.05

```
#inclined plane
angle=0.10016742116
self.RigidBody2Ds[0].radius = 0.5
self.RigidBody2Ds[0].setScale(0.5)
self.RigidBody2Ds[0].setPos(-0.,3,self.RigidBody2Ds[0].radius)
self.RigidBody2Ds[0].vel = Vec3(-0.1,0,-0.9)
self.RigidBody2Ds[0].doGravity = True
self.RigidBody2Ds[0].inverseMomentOfInertia =
self.RigidBody2Ds[0].inverseMass/(2./3.*self.RigidBody2Ds[0].radius
**2)

self.RigidBody2Ds[1].radius = 2.0
self.RigidBody2Ds[1].setScale(2.0)
self.RigidBody2Ds[1].setPos(0.2,3,-self.RigidBody2Ds[1].radius-1.)
self.RigidBody2Ds[1].vel = Vec3(0,0,0)
self.RigidBody2Ds[1].inverseMass = 0.0
self.RigidBody2Ds[1].inverseMomentOfInertia = 0.0

#incline
self.RigidBody2Ds[0].theta = angle
self.RigidBody2Ds[1].theta = angle

if (len(self.RigidBody2Ds)>2):
    i=2
    for r in self.RigidBody2Ds[2:]:
        r.radius = 0.3-i*0.01
        r.setScale(0.3-i*0.01)
        r.inverseMass = 1/0.2
        r.inverseMomentOfInertia =
        r.inverseMass/(2./3.*r.radius**2)

        r.setPos(-0.1+(i%3)*0.05,3,2.0+(i-2)*0.5)
        r.vel = Vec3(-sin(angle),0,-cos(angle))
        r.doGravity = True
        r.theta = angle
        i += 1
```